Laboratorio N°3 a

"FORMATO Q - PUNTO FIJO EN C Y LC-3"

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIAS:

IC221 - Arquitectura de computadoras

DOCENTES:

ING. CARBALLO, Roberto Esteban

ING. BENITEZ, Aldo Javier

ING. SCHUSTER, Roberto Nicolás

ING. ROSSLER, Gustavo Omar

ING. DIAZ, Eduardo Sebastian

ING. FELTAN, Patricia Vanesa

SR. Giménez, Nahuel Joaquín

SR. ZUK, Joaquín

ALUMNOS:

SARI FERNÁNDEZ, Maximiliano Gastón

SOLIS, Carlos Esteban

VILLALBA, Agustin

GRUPO: 4

ÍNDICE

Introducción 4
1. Implementacion en lenguaje de alto nivel (C) 4
1.1. Codigo. 4
1.2. Funcionamiento del programa 7
1.3. Resultados 7
1.4. Conclusion 8
2. Implementacion en Assembly para la LC-3 8
2.1. Introduccion 8
2.2. Funcionamiento del programa 12
2.3. Resultados 13
2.3.1. Compatibilización de formatos Q 13
2.3.2. Multiplicación y ajuste del resultado 13
2.3.3. Rango de la salida 13
CONCLUSIÓN 14
REFERENCIAS BIBLIOGRÁFICAS 14

Introducción

El trabajo se desarrolla en dos etapas. Primero se implementan en lenguaje C las conversiones entre formatos Q y las operaciones de suma, resta, multiplicación y división. Luego se trasladan a Assembly LC-3 los puntos correspondientes a la compatibilización de formatos y la multiplicación.

Como valores de prueba se utilizan 5,25, representado en Q4.11, y 2,50, representado en Q7.8. Estos valores permiten observar el cambio de escala entre formatos y verificar los resultados obtenidos en ambas implementaciones.

1. Implementacion en lenguaje de alto nivel (C)

1.1. Codigo.

El programa recibe los dos valores mediante variables float y realiza las conversiones a enteros escalados. Las operaciones intermedias se realizan con int32_t para evitar desbordamientos durante la multiplicación.

```
// Laboratorio 3-a - Formato Q
void formato(float num1_float, float num2_float)
{
    /* ========================================================
     * 1. Conversión de Punto Flotante a Formato Q
     * ========================================================
     * num1 -> Q4.11 (Escala: 2^11 = 2048)
     * num2 -> Q7.8  (Escala: 2^8  = 256)
     *
     * El valor entero almacenado se obtiene multiplicando
     * el número real por 2^n (donde n son los bits fraccionarios).
     */

    const int32_t escala_q11 = 1 << 11;
    const int32_t escala_q8  = 1 << 8;

    int16_t num1_q11 = (int16_t)(num1_float * escala_q11);
    int16_t num2_q8  = (int16_t)(num2_float * escala_q8);

    /* ========================================================
     * 2. Compatibilización para Suma y Resta
```

```
 * ========================================================
 * Para operar ambos números, deben compartir la misma cantidad
 * de bits fraccionarios. Se eleva num2 de Q7.8 a Qx.11:
 *
 * Diferencia: 11 - 8 = 3 bits -> Se multiplica por 2^3 (<< 3).
 */

int32_t num2_q11 = (int32_t)num2_q8 << 3;


/* ========================================================
 * 3. Operaciones de Suma y Resta
 * ========================================================
 * Ambos operandos están en Qx.11. El resultado se mantiene
 * en la misma representación.
 */

int32_t suma_q11  = (int32_t)num1_q11 + num2_q11;
int32_t resta_q11 = (int32_t)num1_q11 - num2_q11;

// Valores reales (opcional si se requiere recuperar el float):
// float suma_decimal  = (float)suma_q11 / escala_q11;
// float resta_decimal = (float)resta_q11 / escala_q11;


/* ========================================================
 * 4. Multiplicación
 * ========================================================
 * Multiplicar Q4.11 por Q7.8 resulta en un formato Qx.19
 * (11 + 8 = 19 bits fraccionarios).
 */

int32_t mult_q19 = (int32_t)num1_q11 * (int32_t)num2_q8;

/* Para normalizar el resultado de regreso a Qx.8,
 * se desplaza a la derecha 11 posiciones (19 - 8 = 11).
 */
int32_t mult_res_q8 = mult_q19 >> 11;
```

```
    // Valor real del producto:
    // float mult_decimal = (float)mult_res_q8 / escala_q8;


    /* ========================================================
     * 5. División (Conservando máxima resolución)
     * ========================================================
     * Se ajusta el dividendo escalándolo 8 bits más (Qx.11 -> Qx.19)
     * para que al dividir por Qx.8 el resultado quede en Qx.11.
     */

    if (num2_q8 == 0)
    {
        return; // Manejo básico de división por cero
    }

    int32_t dividendo_q19 = (int32_t)num1_q11 << 8;

    // Qx.19 / Qx.8 = Qx.11
    int32_t div_res_q11 = dividendo_q19 / (int32_t)num2_q8;

    // Valor real de la división:
    // float div_decimal = (float)div_res_q11 / escala_q11;
}


int main(void)
{
    float num1_float;
    float num2_float;

    printf("Ingrese el primer numero: ");
    if (scanf("%f", &num1_float) != 1)
    {
        return 1;
    }
```

```
    printf("Ingrese el segundo numero: ");
    if (scanf("%f", &num2_float) != 1)
    {
        return 1;
    }

    formato(num1_float, num2_float);

    return 0;
}}
```

1.2. Funcionamiento del programa

Para el primer valor, 5,25, se utiliza Q4.11. La conversión se obtiene mediante 5,25 × 2048 = 10752, que en hexadecimal corresponde a x2A00. Para el segundo valor, 2,50, se utiliza Q7.8: 2,50 × 256 = 640 = x0280

Valor	Formato	Cálculo	Entero	Hexadecimal
5,25	Q4.11	5,25 × 2048	10752	x2A00
2,50	Q7.8	2,50 × 256	640	x0280

Para sumar y restar, el segundo número se convierte de Q7.8 a Q4.11. Esto se realiza multiplicando 640 por 8, obteniendo 5120. La suma y la resta se ejecutan entonces sobre dos representaciones con la misma escala.

En la multiplicación, 10752 × 640 = 6.881.280. Como los operandos poseen 11 y 8 bits fraccionarios respectivamente, el producto tiene 19 bits fraccionarios. Para expresarlo en Q7.8 se divide por 2048.

En la división se aumenta previamente la resolución del dividendo mediante un desplazamiento equivalente a multiplicar por 2^8. De esta manera se obtiene un resultado con 11 bits fraccionarios, conservando mayor resolución que una división directa entre las representaciones almacenadas.

1.3. Resultados

Operación	Resultado entero	Formato	Valor real
NUM1	10752	Q4.11	5,25
NUM2	640	Q7.8	2,50

NUM2 convertido	5120	Q4.11	2,50
Suma	15872	Q4.11	7,75
Resta	5632	Q4.11	2,75
Multiplicación	3360	Q7.8	13,125
División	≈4300	Q4.11	≈2,10

Los resultados de suma, resta y multiplicación permiten comprobar que las conversiones de escala conservan el valor real de los operandos.

1.4. Conclusion

La implementación en C permitió verificar experimentalmente la representación de números reales mediante formatos Q. Se comprobó que la cantidad de bits fraccionarios determina tanto la resolución como el factor de escala y que las operaciones entre formatos diferentes requieren una conversión previa.

También se verificó que en la multiplicación los bits fraccionarios se acumulan y que posteriormente debe realizarse un ajuste de escala para obtener el formato deseado

2. Implementacion en Assembly para la LC-3

2.1. Introduccion

La segunda parte del laboratorio implementa en Assembly LC-3 los puntos correspondientes a la compatibilización de formatos y la multiplicación. Los valores se incorporan directamente en memoria, ya convertidos a sus formatos Q.

Como la LC-3 trabaja con registros de 16 bits y no dispone de una instrucción nativa de multiplicación ni de desplazamiento, las operaciones se construyen mediante instrucciones básicas como ADD, AND, NOT, LD, ST y saltos condicionales.

```
; ============================================================
; CARGA DE DATOS INICIALES
; ============================================================

    LD R1, NUM1         ; R1 = NUM1 en Q4.11
    LD R2, NUM2         ; R2 = NUM2 en Q7.8


; ============================================================
; COMPATIBILIZACIÓN DE FORMATOS (SUMA Y RESTA)
; ------------------------------------------------------------
; Se eleva NUM2 de Q7.8 a Q4.11 (diferencia de 3 bits: 2^3 = 8).
```

```
; Como LC-3 no desplaza, se multiplica acumulando con ADD.
; ============================================================

        ADD R3, R2, R2          ; R3 = R2 * 2
        ADD R3, R3, R3          ; R3 = R2 * 4
        ADD R3, R3, R3          ; R3 = R2 * 8

        ST R3, NUM2_Q11         ; Guarda NUM2 ajustado a Q4.11


; ============================================================
; OPERACIÓN DE SUMA
; ------------------------------------------------------------
; Ambos operandos están en Q4.11, se suman directamente.
; ============================================================

        ADD R4, R1, R3          ; R4 = NUM1 + NUM2_Q11
        ST R4, SUMA_Q11         ; Guarda el resultado de la suma


; ============================================================
; OPERACIÓN DE RESTA
; ------------------------------------------------------------
; Se obtiene el complemento a dos de NUM2_Q11 y se suma a NUM1.
; ============================================================

        NOT R5, R3              ; R5 = NOT(NUM2_Q11)
        ADD R5, R5, #1          ; R5 = -NUM2_Q11

        ADD R5, R1, R5          ; R5 = NUM1 - NUM2_Q11
        ST R5, RESTA_Q11        ; Guarda el resultado de la resta


; ============================================================
; MULTIPLICACIÓN (Q4.11 * Q7.8 = Q?.19)
; ------------------------------------------------------------
; Se utiliza una representación de doble palabra (base 2^15)
; para evitar desbordamientos en los registros de 16 bits.
```

```
; ============================================================

        LD R1, NUM1             ; R1 = NUM1
        LD R2, NUM2             ; R2 = NUM2

        AND R3, R3, #0          ; R3 = Parte baja del producto (0)
        AND R4, R4, #0          ; R4 = Parte alta del producto (0)

        ADD R5, R2, #0          ; R5 = Contador de iteraciones (NUM2)



; ------------------------------------------------------------
; Bucle de multiplicacion por sumas sucesivas
; ------------------------------------------------------------
MULT_LOOP

        ADD R3, R3, R1          ; producto_bajo += NUM1
        BRn MULT_CARRY          ; Salta si hay desbordamiento (acarreo)
        BRnzp MULT_NEXT

MULT_CARRY
        LD R6, MASK15           ; Carga máscara de 15 bits (x7FFF)
        AND R3, R3, R6          ; Conserva únicamente 15 bits
        ADD R4, R4, #1          ; Incrementa la parte alta

MULT_NEXT
        ADD R5, R5, #-1         ; Decrementa el contador
        BRp MULT_LOOP           ; Repite mientras queden sumas

        ST R3, PRODUCTO_BAJO
        ST R4, PRODUCTO_ALTO



; ============================================================
; AJUSTE DEL PRODUCTO (De Q?.19 a Q7.8)
; ------------------------------------------------------------
; Se eliminan 11 bits excedentes (19 - 8 = 11) dividiendo por 2^11.
; Ecuación: (ALTO * 2^15 + BAJO) / 2048 = (ALTO * 16) + (BAJO / 2048)
```

```
; ============================================================

; Parte 1: ALTO * 16
        ADD R6, R4, R4          ; R6 = ALTO * 2
        ADD R6, R6, R6          ; R6 = ALTO * 4
        ADD R6, R6, R6          ; R6 = ALTO * 8
        ADD R6, R6, R6          ; R6 = ALTO * 16

; Parte 2: BAJO / 2048 (Restas sucesivas)
        ADD R7, R3, #0          ; R7 = PRODUCTO_BAJO
        LD R0, MENOS_2048       ; R0 = -2048

DIV_2048_LOOP
        ADD R7, R7, R0          ; R7 = R7 - 2048
        BRn DIV_2048_FIN        ; Termina si el resultado es negativo
        ADD R6, R6, #1          ; Incrementa la parte entera
        BRnzp DIV_2048_LOOP

DIV_2048_FIN
        ST R6, PRODUCTO_Q8      ; Guarda el producto final en Q7.8


        HALT


; ============================================================
; SECCIÓN DE DATOS Y VARIABLES
; ============================================================

NUM1
        .FILL x2A00             ; 5.25 * 2048 = 10752 (Q4.11)

NUM2
        .FILL x0280             ; 2.50 * 256  = 640   (Q7.8)

NUM2_Q11
        .BLKW 1                 ; NUM2 convertido a Q4.11
```

```
SUMA_Q11
        .BLKW 1             ; Resultado de la suma en Q4.11

RESTA_Q11
        .BLKW 1             ; Resultado de la resta en Q4.11

PRODUCTO_BAJO
        .BLKW 1             ; Parte baja del producto de 32 bits

PRODUCTO_ALTO
        .BLKW 1             ; Parte alta del producto de 32 bits

PRODUCTO_Q8
        .BLKW 1             ; Producto final convertido a Q7.8

MASK15
        .FILL x7FFF         ; Máscara para conservar 15 bits

MENOS_2048
        .FILL xF800         ; -2048 en complemento a dos


        .END
```

2.2. Funcionamiento del programa

Para resolver la exponenciación en la LC-3 sin instrucciones complejas, el programa se divide en etapas lógicas. Inicialmente se cargan NUM1 y NUM2 desde memoria. NUM1 contiene x2A00 correspondiente a 5,25 en Q4.11, mientras NUM2 contiene x0280, correspondiente a 2,50 en Q7.8.

La conversión de Q7.8 a Q4.11 se realiza multiplicando por 8 mediante tres duplicaciones sucesivas. Posteriormente se ejecutan la suma y la resta. La resta utiliza complemento a dos mediante NOT y ADD con 1.

La multiplicación se implementa mediante sumas sucesivas. Debido a que el producto supera los 16 bits, se utiliza una representación interna dividida en una parte alta y una parte baja. Una vez obtenido el producto, se realiza el ajuste de Q?.19 a Q7.8 dividiendo por 2048.

2.3. Resultados

2.3.1. Compatibilización de formatos Q

El valor NUM2 se encuentra en Q7.8 y debe llevarse a Q4.11. La diferencia entre ambos formatos es de tres bits fraccionarios, por lo que se multiplica la representación entera por 2^3.

640 × 8 = 5120 = x1400

5120 / 2048 = 2,50

Variable	Decimal	Hexadecimal	Formato
NUM1	10752	x2A00	Q4.11
NUM2	640	x0280	Q7.8
NUM2_Q11	5120	x1400	Q4.11
SUMA_Q11	15872	x3E00	Q4.11
RESTA_Q11	5632	x1600	Q4.11

2.3.2. Multiplicación y ajuste del resultado

El producto entero es:

10752 × 640 = 6.881.280

La cantidad de bits fraccionarios iniciales es 11 + 8 = 19. Para obtener Q7.8 se deben eliminar 11 bits fraccionarios:

6.881.280 / 2048 = 3360

3360 / 256 = 13,125

Por lo tanto, el resultado final almacenado en PRODUCTO_Q8 es x0D20.

2.3.3. Rango de la salida

El producto entero es:

10752 × 640 = 6.881.280

La cantidad de bits fraccionarios iniciales es 11 + 8 = 19. Para obtener Q7.8 se deben eliminar 11 bits fraccionarios:

6.881.280 / 2048 = 3360

3360 / 256 = 13,125

Por lo tanto, el resultado final almacenado en PRODUCTO_Q8 es x0D20.

CONCLUSIÓN

El desarrollo del laboratorio permitió comprender y aplicar la representación de números reales mediante formatos Q de punto fijo tanto en lenguaje C como en Assembly LC-3.

En la implementación en C se comprobó que el número de bits fraccionarios determina la escala y la resolución de cada representación. También se verificó la necesidad de compatibilizar los formatos antes de realizar operaciones como suma y resta. En la multiplicación, los bits fraccionarios de ambos operandos se suman y posteriormente debe realizarse un ajuste de escala.

En la implementación Assembly se reprodujeron estas operaciones utilizando únicamente las herramientas disponibles en la arquitectura LC-3. La conversión Q7.8 a Q4.11 se realizó mediante duplicaciones con ADD, la resta mediante complemento a dos y la multiplicación mediante sumas sucesivas. Debido a la limitación de los registros a 16 bits, el producto se manejó utilizando más de una palabra antes de realizar la conversión final.

Los resultados obtenidos para el caso de prueba fueron 7,75 para la suma, 2,75 para la resta y 13,125 para la multiplicación, coincidiendo con los resultados matemáticos esperados. De esta manera se verificó el correcto funcionamiento de las conversiones y operaciones implementadas.

REFERENCIAS BIBLIOGRÁFICAS

[1] S. Harris and D. Harris, Digital design and computer architecture: arm edition: Morgan Kaufmann, 2015.

[2] Y. N. Patt and S. J. Patel, Introduction to Computing Systems: From Bits & Gates to C & Beyond, 2nd ed.: McGraw-Hill, 2004.

[3] Guía de Laboratorio N°3 a - Formato Q, Facultad de Ingeniería, UNaM.